Week 3 - Nested stacks vs cross-stack references (Exports / ImportValue)

DOP-C02 Domain 2 (Configuration Management and IaC). Big systems shouldn't live in one giant template. CloudFormation gives **two ways to split** infrastructure across stacks, and the exam tests **which one fits a scenario**:

- **Nested stacks:** a **parent** stack creates **child** stacks as resources. They form **one unit**, deployed and updated together.
- **Cross-stack references: independent** stacks share values through **Exports** and `Fn::ImportValue`. They have **separate lifecycles**.

Part 1 - Why split stacks at all

Reason	Example
Limits	**500 resources** per stack and a **1 MB** template (from S3)
Reuse	The same standard ALB, VPC or RDS template used by many apps
Ownership	The network team owns the VPC; app teams own their services
Lifecycle	The VPC changes once a year, the app every day
Blast radius	A failing app update shouldn't touch the database or network
Speed and clarity	Smaller templates are easier to read, review and test

Part 2 - Nested stacks

How they work

```
# root.yaml (parent)
Parameters:
  Environment: { Type: String, AllowedValues: [dev, prod] }
Resources:
  Network:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: https://my-templates.s3.amazonaws.com/network.yaml   # must be an S3 URL
      Parameters:
        Environment: !Ref Environment
        VpcCidr: 10.0.0.0/16
      TimeoutInMinutes: 30
      Tags:
        - { Key: app, Value: orders }

  Database:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: https://my-templates.s3.amazonaws.com/database.yaml
      Parameters:
        VpcId: !GetAtt Network.Outputs.VpcId                # child output -> sibling parameter
        SubnetIds: !GetAtt Network.Outputs.PrivateSubnetIds  # lists passed as comma-delimited strings

  App:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: https://my-templates.s3.amazonaws.com/app.yaml
      Parameters:
        VpcId: !GetAtt Network.Outputs.VpcId
        DbEndpoint: !GetAtt Database.Outputs.Endpoint

Outputs:
  AppUrl:
    Value: !GetAtt App.Outputs.Url
```

```
# network.yaml (child) must declare Outputs for the parent to read
Outputs:
  VpcId:
    Value: !Ref Vpc
  PrivateSubnetIds:
    Value: !Join [",", [!Ref PrivSubA, !Ref PrivSubB]]
```

Key facts

- **Hierarchy:** the **root stack** contains **parent** and **child** stacks; children can have children.
- `TemplateURL` **must point to S3.** `aws cloudformation package` (or `sam package` / `sam deploy`) **uploads local child templates** (and Lambda code) to S3 and **rewrites** `TemplateURL`; then `aws cloudformation deploy`.
- **Parameters to a child** are **strings**: lists become comma-delimited strings, declared as `CommaDelimitedList` or `List<...>` in the child.
- **Values back:** `!GetAtt ChildStack.Outputs.OutputName`. **Sibling to sibling:** the parent passes child A's output as a parameter to child B, which also creates an **implicit dependency** (A first).
- **No exports needed** between parent and children, so **no export locks**.
- **One unit:** create, update and delete **through the root**. A child failure rolls back the **whole tree**. Deleting the root deletes the children (unless a child has DeletionPolicy Retain).
- **Don't update a nested stack directly**; update **from the root**.
- Each child is a **real stack** with its own events, outputs, drift results and **its own 500-resource limit**: nested stacks **scale past 500 resources**. Every child counts toward the account **stack quota**.
- Child names are generated, for example `orders-prod-Network-1A2B3C4D5E`.
- **Change sets:** `--include-nested-stacks` shows resource-level changes inside children.

- **Termination protection** is set on the **root**.
- **Capabilities:** IAM in any child needs `CAPABILITY_IAM` / `CAPABILITY_NAMED_IAM` on the root operation. Macros, transforms and SAM `AWS::Serverless::Application` need `CAPABILITY_AUTO_EXPAND`.
- **Failures:** read the **nested stack's own events**; the root only says "Embedded stack ... was not successfully created".
- `UPDATE_ROLLBACK_FAILED` **in a child:** `continue-update-rollback` **on the root** with `--resources-to-skip NestedStackName.LogicalId`.
- **Serverless:** `AWS::Serverless::Application` is a nested stack (from the Serverless Application Repository or a local template).

Typical uses

- **Reusable building blocks** (standard ALB, RDS with backups and alarms, security baseline).
- **Splitting one large application** into parts that still deploy together.
- **Getting past the 500-resource limit.**

Part 3 - Cross-stack references (Exports / ImportValue)

```
# network-prod stack (producer)
Outputs:
  VpcId:
    Value: !Ref Vpc
    Export:
      Name: !Sub "${AWS::StackName}-VpcId"          # network-prod-VpcId
  PrivateSubnets:
    Value: !Join [",", [!Ref PrivSubA, !Ref PrivSubB]]
    Export:
      Name: !Sub "${AWS::StackName}-PrivateSubnets"
```

```
# orders-app stack (consumer)
Parameters:
  NetworkStackName: { Type: String, Default: network-prod }
Resources:
  AppSg:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: orders app
      VpcId:
        Fn::ImportValue: !Sub "${NetworkStackName}-VpcId"
  Service:
    Type: AWS::ECS::Service
    Properties:
      NetworkConfiguration:
        AwsvpcConfiguration:
          Subnets: !Split [",", !ImportValue network-prod-PrivateSubnets]
```

Rules (heavily tested)

- **Export names unique within an account and Region**: prefix with the stack name.
- **Same account and same Region only.** No cross-Region or cross-account imports.
- **Export lock:** while **any** stack imports an export, you **can't delete** the exporting stack, and **can't change or remove** that export ("Export ... cannot be updated as it is in use by ..."). Update the **importing stacks first**.
- **Create order:** the producer must exist **before** the consumer, otherwise "No export named ... found".
- `Fn::ImportValue` can build the name with `!Sub` **and parameters**, but **can't** depend on resources in the importing stack.
- **Importing stacks don't update automatically** when values change (and values can't change while imported).
- **Who imports:** `aws cloudformation list-exports` and `list-imports --export-name network-prod-VpcId`, or the console **Exports** page.
- **Lists:** Join on export, `!Split` on import.
- Exports count toward a per-account, per-Region quota; **200 outputs** per template.

Typical uses

- **Layered architecture:** network, then security / shared services, then data, then many app stacks, each **owned and deployed independently**.
- **Shared resources used by many teams:** a VPC, shared ALB, KMS key, logging bucket.

Part 4 - Side-by-side comparison

	Nested stacks	Cross-stack references
Relationship	Parent **owns** children (`AWS::CloudFormation::Stack`)	Independent **peer** stacks
Lifecycle	**One unit**: created, updated, rolled back, deleted **together**	**Separate** schedules
Who deploys	One team or pipeline deploys the root	Different teams or pipelines
Passing values	**Parameters** down, `GetAtt Child.Outputs.X` up	`Export` + `Fn::ImportValue`
Coupling	Tight; updates through the root	Loose deployment, **hard lock** on exports in use
Reuse	**Same template** reused in many parents (each gets its own resources)	**One shared instance** used by many stacks
Changing a shared value	Update the root; children get new parameters	**Blocked while imported**
Deleting	Delete the root, children go	Producer can't be deleted while imported
Limits	Each child has its own 500 resources	200 outputs, export quota, unique names
Cross-Region / account	No	**No**: use SSM, StackSets, custom resources
Failure scope	Child failure rolls back the **whole tree**	Only the stack being deployed

- **Nested = components of ONE application, deployed together, and reuse of a template.**
- **Cross-stack = SHARED resources with their own lifecycle, and reuse of a deployed resource.**

Part 5 - Decision guide

Scenario	Choose
Network team manages a VPC used by 15 app teams on different schedules	**Cross-stack exports** (or SSM parameters)
A standard ALB + target group + alarms setup reused in every app template	**Nested stack** (or a CloudFormation **module**)
One application has 700 resources	**Nested stacks**
The database must not be redeployed or rolled back when the app tier fails	**Separate stacks** (cross-stack), not nested in the app root
Value needed in **another Region or account**	**SSM Parameter Store** (per Region / RAM sharing), **StackSets**, or a **custom resource**
Change a shared value without export locks	**SSM Parameter Store** (no lock, but no protection)
Whole app must roll back together if any part fails	**Nested stacks**
Same baseline stack in many accounts and Regions	**StackSets**

SSM Parameter Store as the third option

```
# producer writes the value
VpcIdParam:
  Type: AWS::SSM::Parameter
  Properties:
    Name: /shared/network/prod/vpc-id
    Type: String
    Value: !Ref Vpc

# consumer reads it (no export lock)
Parameters:
  VpcId:
    Type: AWS::SSM::Parameter::Value<AWS::EC2::VPC::Id>
    Default: /shared/network/prod/vpc-id
```

- **Pros:** no export lock; works for **non-CloudFormation consumers** (scripts, Lambda, Terraform); written per Region with StackSets; advanced parameters **shareable across accounts with AWS RAM**.
- **Cons: nothing stops** the producer deleting the VPC while consumers use it; consumers pick up new values only on their **next stack update**.

Part 6 - Common patterns and tools

- **Layered stacks:** `network`, `security` (IAM roles, KMS keys), `data` (RDS, buckets with Retain), `app-*`. Lower layers **export**, higher layers **import**. Deploy bottom to top; delete top to bottom.
- **Root stack per app with nested components** (`alb`, `ecs-service`, `alarms` children) that still **imports** the shared VPC. Mixing both is normal.
- **CloudFormation modules** (Registry, used inline as `MyOrg::Web::Alb::MODULE`): reuse **without separate stacks**.
- **CDK:** passing a resource between CDK stacks **automatically creates Export / ImportValue**, which causes the classic "can't remove the resource from the producer while it's imported" problem (deploy in two steps or use SSM). CDK `NestedStack` creates nested stacks.
- **CodePipeline:** deploy layered stacks in **ordered stages or actions**; pass outputs with **output variables** or imports. For nested stacks, **package** in CodeBuild first, then deploy with change sets.

Part 7 - Exam traps

Trap	Truth
ImportValue from another Region or account	**Same account and Region only.** SSM Parameter Store, StackSets or custom resource
Delete the network stack while app stacks import its exports	**Blocked.** Remove imports first
Change an exported value with a simple update	**Blocked while imported**
Two stacks both export `VpcId`	**Unique per account and Region.** Prefix with stack name
Consumer fails: No export named X found	Producer not deployed yet, other Region, or wrong name
Pass values between nested siblings with exports	**Not needed:** `!GetAtt ChildA.Outputs.X` as a parameter to child B
`TemplateURL` as a local file path	Must be **S3**: `aws cloudformation package`
Update a child nested stack directly	Update **through the root**
Root shows Embedded stack failed: where's the error?	**Nested stack's own events**
Nested stacks let each team deploy its part independently	**No**, deployed together through the root. Use cross-stack
A child failure only rolls back that child	**The whole root update rolls back**
Stack needs more than 500 resources	**Nested stacks**
Termination protection on a nested stack	Set it on the **root**
Nested SAM application or macros fail on capabilities	`CAPABILITY_AUTO_EXPAND` (+ `CAPABILITY_IAM`)
Reuse a standard ALB setup in every app	**Nested stack** (or module). Exports share one instance, not a template
Avoid export locks for a frequently changing shared value	**SSM Parameter Store**
Lists in exports or nested parameters	Strings only: **Join** and **Split**
Nested stack stuck in UPDATE_ROLLBACK_FAILED	`continue-update-rollback` **on the root** with `--resources-to-skip Child.Resource`

Part 8 - Hands-on lab (about 60 to 90 minutes, low cost; no NAT gateways)

Lab 1 - Cross-stack exports

`network.yaml`:

```
Resources:
  Vpc:
    Type: AWS::EC2::VPC
    Properties: { CidrBlock: 10.20.0.0/16 }
  SubA:
    Type: AWS::EC2::Subnet
    Properties: { VpcId: !Ref Vpc, CidrBlock: 10.20.1.0/24, AvailabilityZone: !Select [0, !GetAZs ""] }
  SubB:
    Type: AWS::EC2::Subnet
    Properties: { VpcId: !Ref Vpc, CidrBlock: 10.20.2.0/24, AvailabilityZone: !Select [1, !GetAZs ""] }
Outputs:
  VpcId:
    Value: !Ref Vpc
    Export: { Name: !Sub "${AWS::StackName}-VpcId" }
  Subnets:
    Value: !Join [",", [!Ref SubA, !Ref SubB]]
    Export: { Name: !Sub "${AWS::StackName}-Subnets" }
```

`app.yaml`:

```
Parameters:
  NetworkStack: { Type: String, Default: lab-network }
Resources:
  AppSg:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: lab app
      VpcId: { "Fn::ImportValue": !Sub "${NetworkStack}-VpcId" }
Outputs:
  FirstSubnet:
    Value: !Select [0, !Split [",", { "Fn::ImportValue": !Sub "${NetworkStack}-Subnets" }]]
```

1. Deploy `app.yaml` **first**: fails with "No export named lab-network-VpcId found".
2. Deploy `network.yaml` as `lab-network`, then `app.yaml` as `lab-app`.
3. `aws cloudformation list-imports --export-name lab-network-VpcId` shows `lab-app`.
4. Change the VPC CIDR (replaces the VPC, new VpcId) and deploy: **fails**, export in use.
5. Try to delete `lab-network`: **fails** for the same reason.

Lab 2 - The same with SSM Parameter Store

1. Add an `AWS::SSM::Parameter` `/lab/network/vpc-id` to the network stack.
2. Change the app to parameter type `AWS::SSM::Parameter::Value<AWS::EC2::VPC::Id>` with that default, remove ImportValue, redeploy.
3. The export can now be removed. Note the risk: nothing stops the VPC being deleted while the app uses it.

Lab 3 - Nested stacks with package

1. Folder with `root.yaml`, `network-child.yaml` (same VPC, **no Export**, outputs only) and `sg-child.yaml` (one SG with a `VpcId` parameter and output `SgId`).
2. In `root.yaml`, `TemplateURL: network-child.yaml` and `TemplateURL: sg-child.yaml` (local paths); pass `!GetAtt Network.Outputs.VpcId` to the SG child.
3. Package and deploy:

```
aws cloudformation package --template-file root.yaml --s3-bucket <your-bucket> --output-template-file
    packaged.yaml
aws cloudformation deploy --template-file packaged.yaml --stack-name lab-root
```

4. `packaged.yaml` now has S3 URLs; the console shows **3 stacks** (root plus 2 generated child names).

Lab 4 - Change sets and failures in nested stacks

1. Change the SG description in `sg-child.yaml`, package, and create a change set **with** `--include-nested-stacks`: the SG change shows inside the child.
2. Put an invalid CIDR in `network-child.yaml` and deploy: the root shows "Embedded stack ... failed". Open the **child's events** for the real error; the **whole tree rolls back**.
3. Enable termination protection on `lab-root`, try to delete (blocked), then disable it.

Clean up

- Delete `lab-app`, then `lab-network`, then `lab-root` (children go too), the SSM parameter, and the packaged templates in S3.

Part 9 - Check yourself

Questions

1. What resource type creates a nested stack, and what property points to the child template?
2. How does a parent read a value from a child? How do two sibling children share a value?
3. The root template uses local paths in `TemplateURL`. How do you deploy it?
4. A root update fails with "Embedded stack ... was not successfully updated". Where is the cause?
5. Which approach for a VPC used by many independently deployed app stacks? Why?
6. Which approach to reuse a standard ALB setup in many app templates? Why?
7. An application needs 800 resources. How do you handle the 500-resource limit?
8. A stack in eu-west-1 needs a value exported in us-east-1. Options?
9. You must replace the shared VPC, but the update fails because the export is in use. Process?
10. App stacks fail with "No export named network-prod-VpcId found". Two likely causes?
11. How do you find which stacks import a given export?
12. Main advantage and main risk of SSM Parameter Store instead of exports?
13. A child stack is stuck in `UPDATE_ROLLBACK_FAILED`. How do you recover?
14. Can each team deploy its own child stack independently in a nested design?
15. Which capability is needed when a nested template uses a macro or a SAM serverless application?

Answers

1. `AWS::CloudFormation::Stack` with `TemplateURL` (an S3 URL) and optional `Parameters`.
2. `!GetAtt ChildStack.Outputs.OutputName`. Siblings: the parent passes child A's output as a **parameter** to child B.
3. `aws cloudformation package` (uploads children to S3, rewrites URLs), then `aws cloudformation deploy`.
4. In the **nested stack's own events**.
5. **Cross-stack references** (or SSM parameters): the VPC has its **own lifecycle and owner**; consumers deploy independently.
6. **Nested stack** (or module): reuse a **template** as a component deployed with each app.
7. **Split into nested stacks**; each child has its own 500-resource limit.
8. **Not ImportValue.** SSM Parameter Store in eu-west-1 (StackSets, pipeline or custom resource), a cross-Region **custom resource**, or a parameter.
9. Update every **importing stack** to stop using the export (or use a new export), update or replace the VPC and its export, then point consumers to the new value.
10. **Producer not deployed** (or failed), **different Region or account**, or **wrong export name**.
11. `aws cloudformation list-imports --export-name <name>` or the console Exports page.
12. **Advantage: no export lock**, works across tools (and accounts with RAM sharing). **Risk: no protection**; the producer can delete or change the resource, and consumers update only on their next deployment.
13. `continue-update-rollback` **on the root** with `--resources-to-skip ChildStackLogicalId.ResourceLogicalId`.
14. **No.** Nested stacks deploy **through the root as one unit**; use separate stacks for independent teams.
15. `CAPABILITY_AUTO_EXPAND` (plus `CAPABILITY_IAM` / `CAPABILITY_NAMED_IAM` for IAM resources).

Quick memory summary

- **Nested:** `AWS::CloudFormation::Stack` + S3 `TemplateURL` (**package** for local files); parameters down (strings), `GetAtt Child.Outputs.X` up; one unit through the root; whole-tree rollback; each child 500 resources; termination protection and continue-update-rollback on the root; `--include-nested-stacks`; `CAPABILITY_AUTO_EXPAND` for macros / SAM apps.
- **Cross-stack:** `Export` + `Fn::ImportValue`; names unique per account + Region; **same account and Region only**; **locked while imported**; producer first; `list-imports`; Join / Split for lists.
- **Choose:** nested = components of one app / template reuse / past 500; cross-stack = shared resources with their own lifecycle; **SSM Parameter Store** = no lock, cross-Region / account, but no protection.